

REQ1 Design Rationale — Teleportation (Door, Circle, Cube)

Introduction

REQ1 adds three teleportation sources—Tele-Door (#), Teleportation Circle (0), and a Teleport Cube plus environmental fire and stacking burn DoT. The solution must support intra-map and inter-map jumps, 50% cube malfunction, burn persistence and stacking, Fire→Dirt permanence, and integrate cleanly with the FIT2099 engine (no engine edits, correct use of Actions/Ground/Item).

Design A — Action-Centric Teleportation (Chosen)

Idea: Centralize all teleport logic in a reusable `TeleportAction`. Teleport sources (Door/Circle/Cube) are thin; they only expose actions from `allowableActions(...)` and configure behavior via parameters/flags. Environmental burn is handled by a `BurningManager` and `Fire` ground.

Key classes

- `TeleportAction` (destination selection, cube 50/50, safe landing, burn triggers)
- `TeleDoor`, `TeleportationCircle` (Grounds) → build labeled `TeleportActions`
- `TeleportCube` (Item) → exposes one “Use Teleport Cube” action
- `TeleportDestination` (value object)
- `BurningManager` (stacking DoT), `Fire` (temporary ground), `Dirt` (permanent)

Pros

- **SRP/DRY:** One place for teleport rules (malfunction, safe landing, inter/intra selection). Minimal duplication across sources.

- **OCP:** Add a new teleporter by composing a `TeleportAction`—no edits to existing teleport code.
- **Integration:** Uses engine's **Action** pathways; Grounds/Item remain declarative. No engine modification.
- **Safety & robustness:** `safeDestination` & `randomFreeLocation` prevent “move onto actor” crashes.
- **KISS:** `TeleportDestination` is a simple (map,x,y); cube “other-map” success is a direct predicate (`dest.map != currentMap`).
- **Connascence (name & meaning):** Clear menu labels (“Teleport via Door to <Map> (x,y)”). Flags (`burnAroundSource`, `cube50Malfunction`, `forcedIndex`) make behavior explicit.
- **Testability:** `TeleportAction` can be unit-tested in isolation.

Cons

- `TeleportAction` grows with features → must be kept cohesive (guard against “god action”).
- Requires discipline to keep source-specific quirks (door vs circle burns) outside the action or behind small helpers.

Design B — Per-Source Logic (Door/Circle/Cube own their teleports)

Idea: Implement teleport behavior inside each source (`TeleDoor.executeTeleport(...)`, `TeleportationCircle.executeTeleport(...)`, `TeleportCube.executeTeleport(...)`). Each class performs movement, burn, and safety checks itself.

Pros

- Behavior is **local** to each source; easy to read “in place”.

- Slightly fewer constructor parameters/flags.

Cons

- **Violates DRY:** Three similar implementations for movement, safety, inter/intra choice, and messages.
- **Higher change cost:** Any tweak (e.g., safe-landing rule) requires 3 edits; easy to diverge or miss an edge case.
- **Weaker extensibility (OCP):** Adding a new teleporter means copying logic again.
- **Risk of inconsistency:** Door vs Circle vs Cube may drift (e.g., different bug fixes).
- **Harder to test:** Duplicated behavior increases test surface and failure modes.
- **Connascence of algorithm:** Teleport rules scattered; harder for teammates to locate the single source of truth.

Why Design A was chosen

Design A maximizes marks across **Implementation Quality (SOLID/DRY)** and **Integration**. It respects the engine's intended extension points (Ground/Item supply **Actions**; environment effects via **Ground.tick** and a **manager**), avoids **instanceof**/downcasting, and keeps all teleport rules **cohesive and reusable**. The explicit flags and labeled actions make intent clear (good connascence) and the centralized safety logic prevents runtime errors. This yields a **clean, extensible** architecture: new teleporters or rule tweaks touch **one** class (**TeleportAction**) or a tiny helper, while Doors/Circles/Cube remain thin and maintainable.